

Competizioni Kaggle

Come approcciarle per la prima volta

Pietro Venturini · Università degli Studi di Brescia · 2021

Cosa è Kaggle?

- 2010** ● Kaggle nasce come piattaforma per organizzare competizioni in ambito ML
- 2017** ● Google acquista Kaggle
- Oggi** ● Competizioni ML
Condivisione di dataset pubblici
Condivisione di Kernel (codice/notebooks)
Kaggle learn

Caso di Studio

Kaggle: Predict Future Sales

Prevedere n° unità vendute in totale nel mese di Novembre 2015 per coppie (item, shop)

sales_train (~3M)

	date	date_block_num	shop_id	item_id	item_price	item_cnt_day
0	02.01.2013	0	59	22154	999.00	1.0
1	03.01.2013	0	25	2552	899.00	1.0
2	05.01.2013	0	25	2552	899.00	-1.0
3	06.01.2013	0	25	2554	1709.05	1.0
4	15.01.2013	0	25	2555	1099.00	1.0

item_categories

	item_category_name	item_category_id
0	PC - Гарнитуры/Наушники	0
1	Аксессуары - PS2	1
2	Аксессуары - PS3	2
3	Аксессуары - PS4	3

items

	item_name	item_id	item_category_id
22160	ЯРМАРКА ТЩЕСЛАВИЯ (Регион)	22160	40
22161	ЯРОСЛАВ. ТЫСЯЧУ ЛЕТ НАЗАД э (BD)	22161	37
22162	ЯРОСТЬ	22162	40
22163	ЯРОСТЬ (регион)	22163	40
22164	ЯРОСТЬ (BD)	22164	37

shops

	shop_name	shop_id
2	Адыгея ТЦ "Мега"	2
3	Балашиха ТРК "Октябрь-Киномир"	3
4	Волжский ТЦ "Волга Молл"	4
5	Вологда ТРЦ "Мармелад"	5

Exploratory Data Analysis

Valori mancanti

NaN

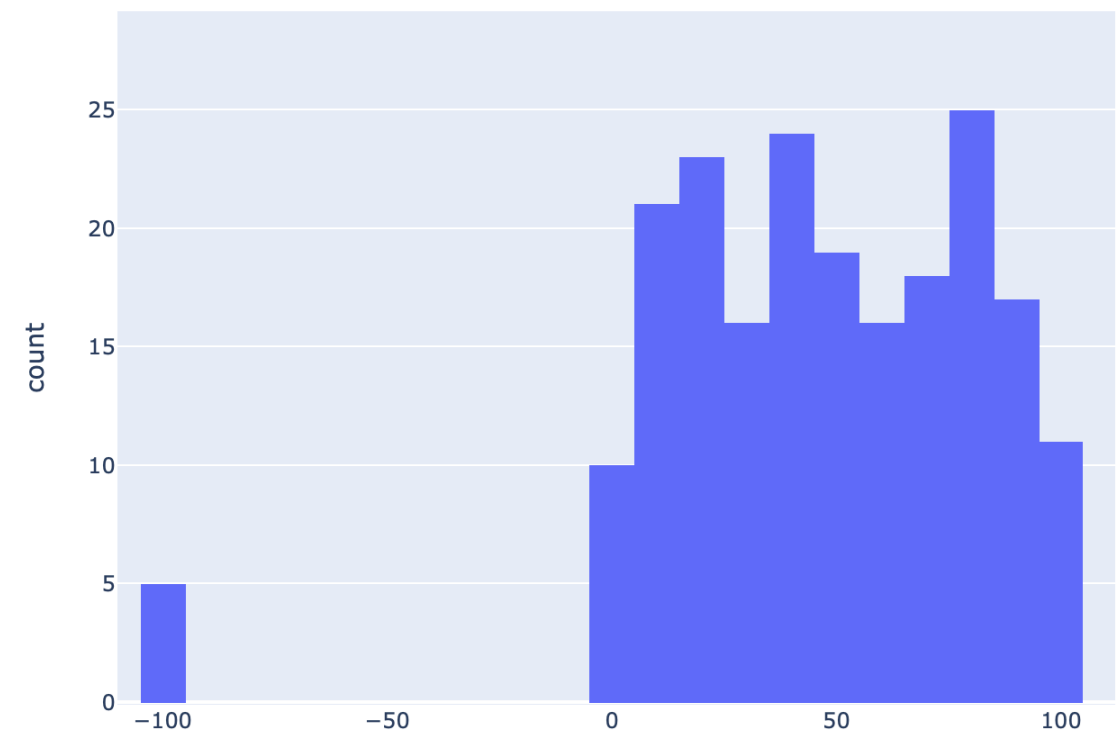
```
df.isna().sum()  
df[df.isna().any(axis=1)]
```

“ ”

```
np.where(df.applymap(lambda x: x == ' '))
```

-1, -999

```
df.describe()  
df['col_name'].hist(bins=30)
```



Exploratory Data Analysis

Valori mancanti

```
[ ] # remove duplicated rows
train = train.drop_duplicates()

# view information about prices and sold quantities
train[['item_price', 'item_cnt_day']].describe()
```

	item_price	item_cnt_day
count	2.935843e+06	2.935843e+06
mean	8.906237e+02	1.242641e+00
std	1.726442e+03	2.612216e+00
min	-1.000000e+00	-2.200000e+01
25%	2.490000e+02	1.000000e+00
50%	3.990000e+02	1.000000e+00
75%	9.990000e+02	1.000000e+00
max	3.079800e+05	2.169000e+03

Exploratory Data Analysis

Valori mancanti

```
[ ] # items with price <= 0
neg_items = train[train.item_price <= 0]
neg_items
```

	date	date_block_num	shop_id	item_id	item_price	item_cnt_day
484683	15.05.2013	4	32	2973	-1.0	1.0

Quanto costava lo stesso prodotto quel giorno in altri negozi?

```
[ ] train[(train['item_id'] == neg_items.loc[0, 'item_id']) &
         (train['date'] == neg_items.loc[0, 'date'])]
```

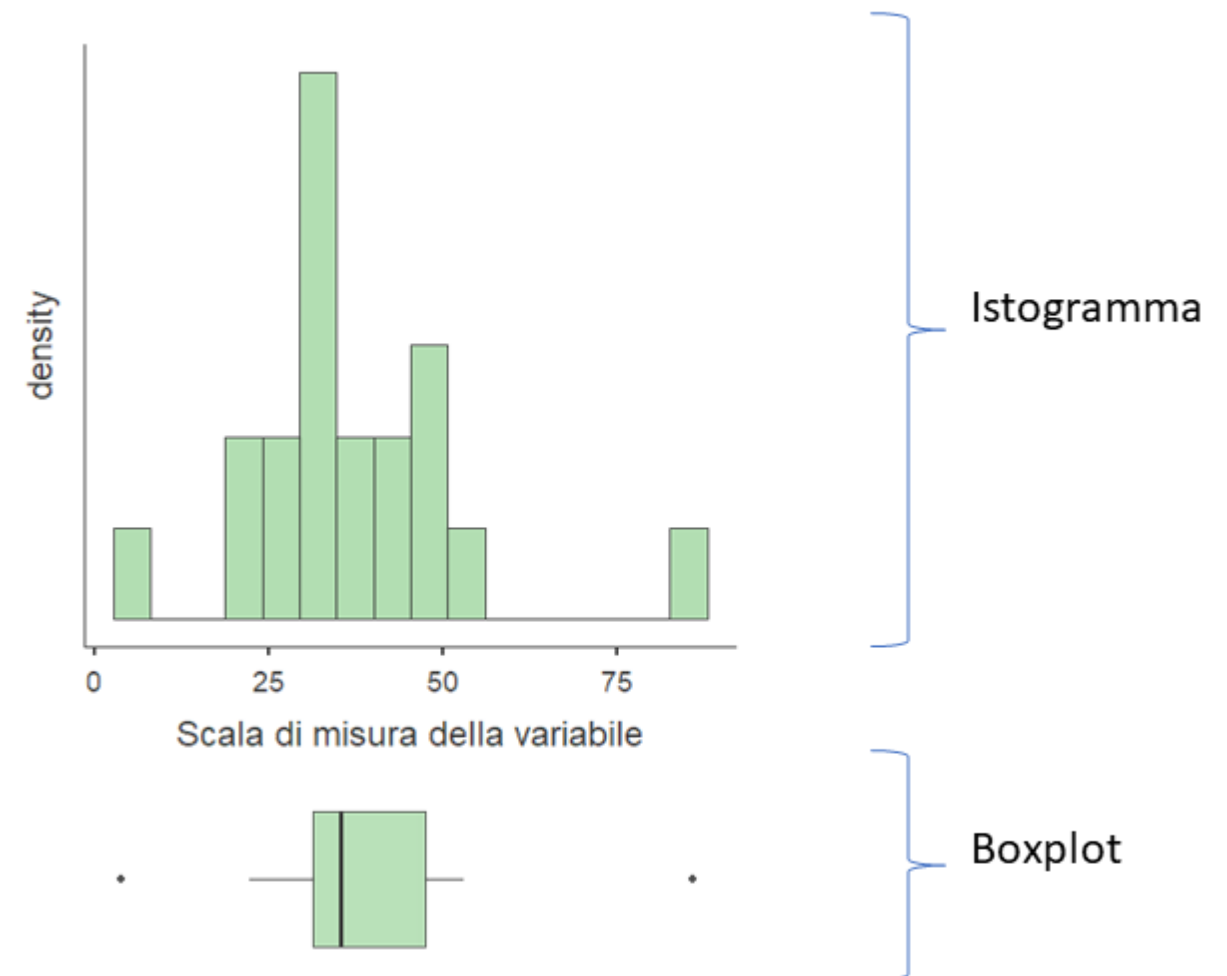
	date	date_block_num	shop_id	item_id	item_price	item_cnt_day
453034	15.05.2013	4	25	2973	1249.5	1.0
469026	15.05.2013	4	46	2973	1249.5	1.0
469951	15.05.2013	4	45	2973	1249.0	1.0
484683	15.05.2013	4	32	2973	-1.0	1.0

Exploratory Data Analysis

Outliers

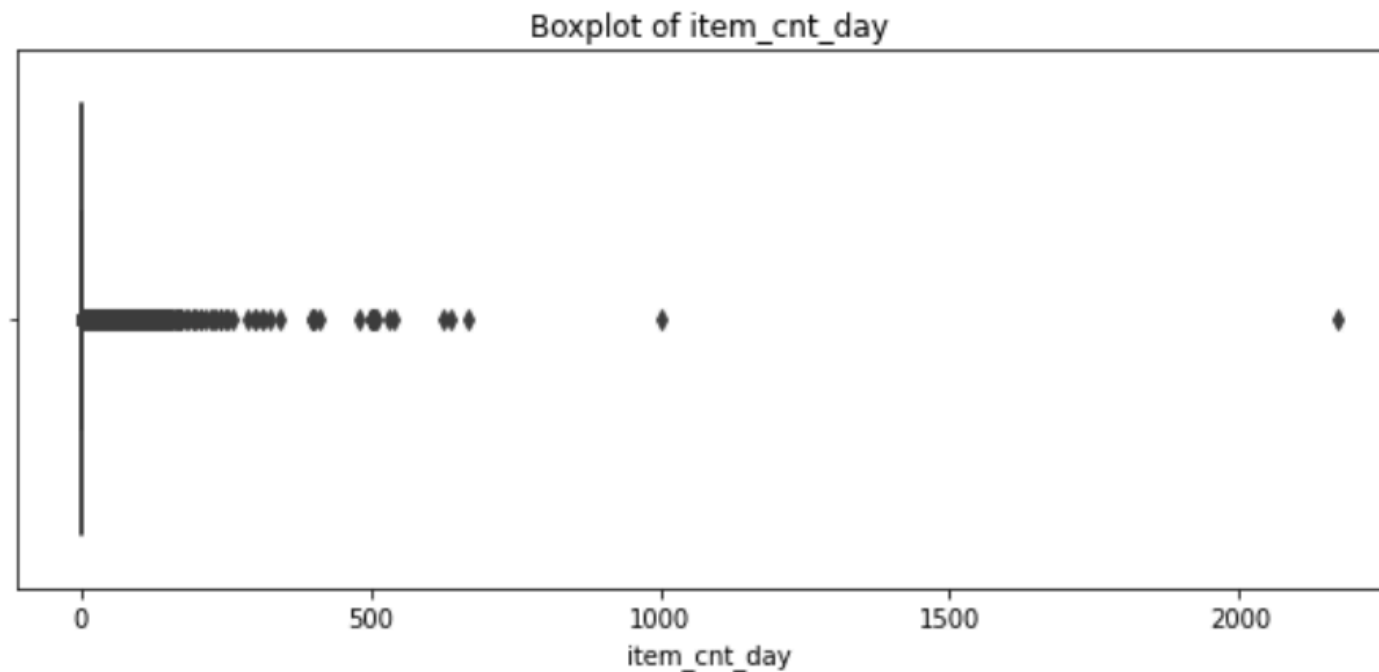
Aiutarsi con dei plot, ad esempio:

- Istogrammi
- BoxPlot



Exploratory Data Analysis

Outliers



- Calcolo q-percentile e limite range di valori

```
upper_bound = np.percentile(train['item_cnt_day'], 99)
train['item_cnt_day'].clip(0, upper_bound, inplace=True)
```

- Oppure rimuovo in base a condizione

```
train = train[train.item_cnt_day < upper_bound]
```

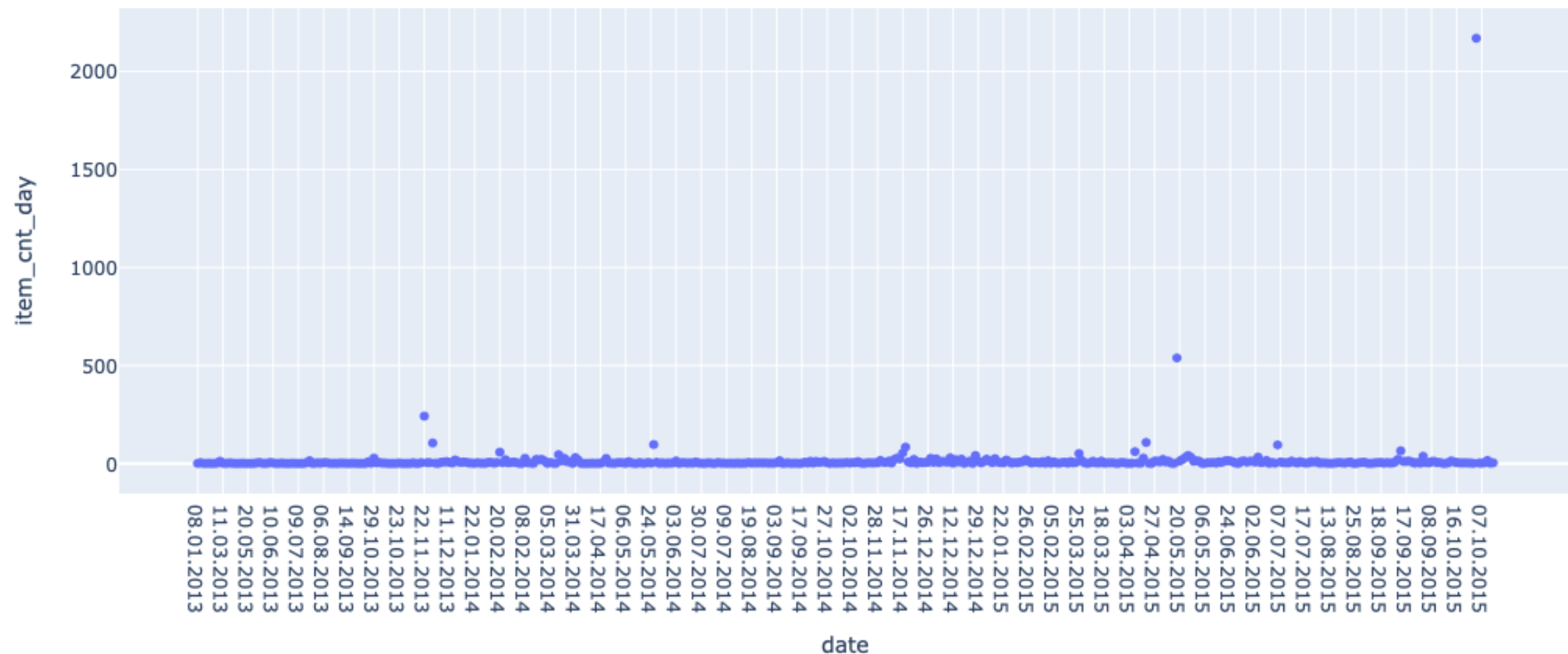

Exploratory Data Analysis

Outliers

- Indagare sul **perché** siano considerati degli outliers

```
id = train.iloc[train.item_cnt_day.idxmax()]['item_id']
```

Quantities of item #11373 (Boxberry delivery) sold every day



Feature engineering

Costruiamo nuove features

- Utilizzare la conoscenza di dominio per creare nuove features
- Features in base al tipo di variabili in questione

Numeriche

	price	price_decimal_part
0	9.99	99
1	4.00	0
2	4.95	95
3	6.99	99

Temporal

	date	month	days_till_christmas
0	12-12-19	12	13
1	02-10-19	10	84
2	28-11-18	11	27
3	23-12-19	12	2

Spaziali

	x	y	dist_from_hospital	avg_price_within_2km
0	240	19	12	931.4
1	102	184	50	800.2
2	-123	-91	77	777.2
3	65	-4	24	829.9

Feature engineering

Negozi

	shop_name	shop_id	shop_city	shop_type	shop_first_month	shop_last_month
2	адыгея тц мега	2	адыгея	тц	0	33
3	балашиха трк октябрькиномир	3	балашиха	трк	0	33
4	волжский тц волга молл	4	волжский	тц	0	33
5	вологда трц мармелад	5	вологда	трц	1	33
6	воронеж плехановская	6	воронеж	NaN	0	33

```
# Estraggo città dal nome del negozio (prima parola)
shops['shop_city'] = shops['shop_name'].str.split(' ').map(lambda x: x[0])
```

```
# Aggiungo il primo mese di ciascun negozio
shops = shops.join(
    train.groupby('shop_id')['date_block_num'].min()
).rename(columns={'date_block_num': 'shop_first_month'})
```

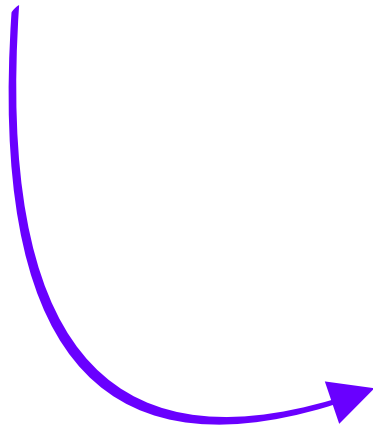
```
# Aggiungo l'ultimo mese di ciascun negozio
shops = shops.join(
    train.groupby('shop_id')['date_block_num'].max()
).rename(columns={'date_block_num': 'shop_last_month'})
```

Feature engineering

Categorie di prodotti

	item_category_name	item_category_id
0	PC - Гарнитуры/Наушники	0
1	Аксессуары - PS2	1
2	Аксессуары - PS3	2
3	Аксессуары - PS4	3
4	Аксессуары - PSP	4
5	Аксессуары - PSVita	5
6	Аксессуары - XBOX 360	6
7	Аксессуары - XBOX ONE	7

	item_category_id	section	name	sub_name	sec_sub_name
0	0	PC	Гарнитуры/Наушники	NaN	NaN
1	1	accessories	PS2	PS	2
2	2	accessories	PS3	PS	3
3	3	accessories	PS4	PS	4
4	4	accessories	PSP	PS	P
5	5	accessories	PSVita	PS	Vita
6	6	accessories	XBOX 360	XBOX	360
7	7	accessories	XBOX ONE	XBOX	ONE



Feature engineering

Prodotti

Prima

	item_name	item_id	item_category_id
5012	NEED FOR SPEED: ЖАЖДА СКОРОСТИ	5012	40
5013	NEED FOR SPEED: ЖАЖДА СКОРОСТИ (3D BD)	5013	38
5014	NEED FOR SPEED: ЖАЖДА СКОРОСТИ (BD)	5014	37
5217	Need For Speed: ProStreet [PC, Цифровая версия]	5217	31
5218	Need For Speed: SHIFT [PC, Цифровая версия]	5218	31
...	...	...	...
5275	Need for Speed: Most Wanted (a Criterion Game)...	5275	22
5276	Need for Speed: Most Wanted (a Criterion Game)...	5276	19
5277	Need for Speed: Most Wanted (a Criterion Game)...	5277	23
5278	Need for Speed: Most Wanted 5-1-0 (Essentials)...	5278	21
5279	Need for Speed: Undercover [PC, Цифровая версия]	5279	31

```
items[items.item_name.str.lower().str.startswith('need for speed')]
```

Feature engineering

Prodotti

Dopo



	item_name	item_id	item_category_id	item_name_2	item_name_3
5012	need for speed жажда скорости	5012	40	0	0
5013	need for speed жажда скорости	5013	38	0	3d bd
5014	need for speed жажда скорости	5014	37	0	bd
5217	need for speed prostreet	5217	31	pc цифровая версия	0
5218	need for speed shift	5218	31	pc цифровая версия	0
...	...	...	...	...	...
5275	need for speed most wanted	5275	22	ps vita русская версия	a criterion game
5276	need for speed most wanted	5276	19	ps3 русская версия	a criterion game
5277	need for speed most wanted	5277	23	xbox 360 русская версия	a criterion game
5278	need for speed most wanted 5 1 0	5278	21	psp английская версия	essentials
5279	need for speed undercover	5279	31	pc цифровая версия	0

Feature engineering

Nomi

Obiettivo: estrarre da nomi simili (e.g. “Need for Speed Most Wanted”, “Need for Speed Prostreet”) l’informazione che li accomuna.



Titolo



Capitolo

Distanza tra stringhe: idealmente vogliamo che stringhe simili abbiano una distanza minore rispetto a stringhe diverse.

Levenshtein

Jaccard

Similarità del coseno

Tante altre...

Per approfondire: <https://www.baeldung.com/cs/string-similarity-edit-distance>

Feature engineering

Nomi

Distanza di Jaccard:

Indice di Jaccard

$$J_{\delta}(X_1, X_2) = 1 - J(X_1, X_2) = 1 - \frac{|X_1 \cap X_2|}{|X_1 \cup X_2|}$$

$$\frac{|\{\text{need, for, speed}\}|}{|\{\text{need, for, speed, prostreet, most, wanted}\}|} = \frac{3}{6} = 0.5$$

$$\frac{|\{\}\}|}{|\{\text{need, for, speed, prostreet, fifa, 17}\}|} = \frac{0}{6} = 0$$

Problema: a parità di cardinalità dell'intersezione, più due stringhe sono lunghe, maggiore sarà la cardinalità dell'unione, e quindi maggiore sarà la loro distanza.

Considero solo le prime **k** parole.

Feature engineering

Nomi

1. Calcolo matrice delle distanze

```
def jaccard(x, y):  
    """Return jaccard distance between two sets of string elements  
    e.g. x = {'the', 'first', 'string'}, y = {'another',  
    'string'}."""  
    return 1 - len(x.intersection(y)) / len(x.union(y))  
  
# consider only first 3 words of item_name  
item_names = [set(x.split()[:3]) for x in items['item_name']]  
d = np.zeros((len(item_names), len(item_names)))  
for i in range(len(item_names)):  
    for j in range(i, len(item_names)):  
        d[j,i] = d[i,j] = jaccard(item_names[i], item_names[j])
```

Feature engineering

Nomi

2. Applico un algoritmo di clustering (e.g. DBSCAN)

```
from sklearn.cluster import dbSCAN  
dbs = dbSCAN(d, metric='precomputed', eps=0.6, min_samples=1)
```

3. L'algoritmo, per ogni punto (per ogni nome di prodotto) mi restituirà l'indice del cluster cui questo è stato assegnato.

4. Uso tale indice per effettuare LabelEncoding (sostituisco al nome del prodotto l'etichetta assegnata).

123

- Need for Speed Prostreet
 - Need for Speed Most Wanted
- Speed for Need...

111

- FIFA 15
- FIFA 14
- FIFA 13

Feature engineering

Vendite

Training set: vendite quotidiane (giorno, shop, item)

Test set: vendite mensili di Novembre (shop, item)

Approccio 1: per ogni coppia (shop, item) stimo un modello lineare in grado di cogliere struttura in media: AR, MA, ARMA, ARIMA, SARIMA...

Problemi:

```
[ ] print("Power set (date x shop x item) size is",  
        len(train.date.unique()) * len(train.shop_id.unique()) * len(train.item_id.unique()))
```

```
Power set (date x shop x item) size is 1262654624
```

- Non teniamo conto di informazioni derivanti da altri items.

Feature engineering

Vendite

Training set: vendite quotidiane (giorno, shop, item)

Test set: vendite mensili di Novembre (shop, item)

Approccio 2: aggreghiamo dati per mese, per ogni coppia (shop, item)

Problemi:

```
[ ] print("Power set (date_block_num × shop × item) size is",  
        len(train.date_block_num.unique()) * len(train.shop_id.unique()) * len(train.item_id.unique()))
```

```
Power set (date_block_num × shop × item) size is 41518624
```

Feature engineering

Vendite

Approccio 3: aggreghiamo dati per mese, per ogni coppia (shop, item) tale per cui vi sia almeno un record in tale mese

```
[ ] # keep in train only sales associated to shops that are also in the test set
train = train.merge(test[['shop_id']].drop_duplicates(), how='inner')

from itertools import product

# create grid of (date_block_num, shop_id, item_id) triplets from train set
grid = []

for block_num in train['date_block_num'].unique():
    mthly_shops = train[train.date_block_num == block_num]['shop_id'].unique()
    mthly_items = train[train.date_block_num == block_num]['item_id'].unique()
    grid.append(np.array(list(product(*[mthly_shops, mthly_items, [block_num]])), dtype='int32'))

sales = pd.DataFrame(np.vstack(grid), columns=['shop_id', 'item_id', 'date_block_num'], dtype=np.int32)

# get monthly sales for each (shop, item) pair in train set
mthly_sales = train.groupby(['shop_id', 'item_id', 'date_block_num'])['item_cnt_day'].sum()
                    .reset_index().rename(columns={'item_cnt_day': 'item_cnt_month'})
sales = pd.merge(sales, mthly_sales, how='left', on=['shop_id', 'item_id', 'date_block_num']).fillna(0)
sales['item_cnt_month'] = sales['item_cnt_month'].astype(int)

# get monthly average price for each (shop, item) pair in train set
avg_prices = train.groupby(['shop_id', 'item_id', 'date_block_num'])['item_price'].mean()
                    .reset_index().rename(columns={'item_price': 'item_avg_price'})
sales = pd.merge(sales, avg_prices, how='left', on=['shop_id', 'item_id', 'date_block_num']).fillna(0.)

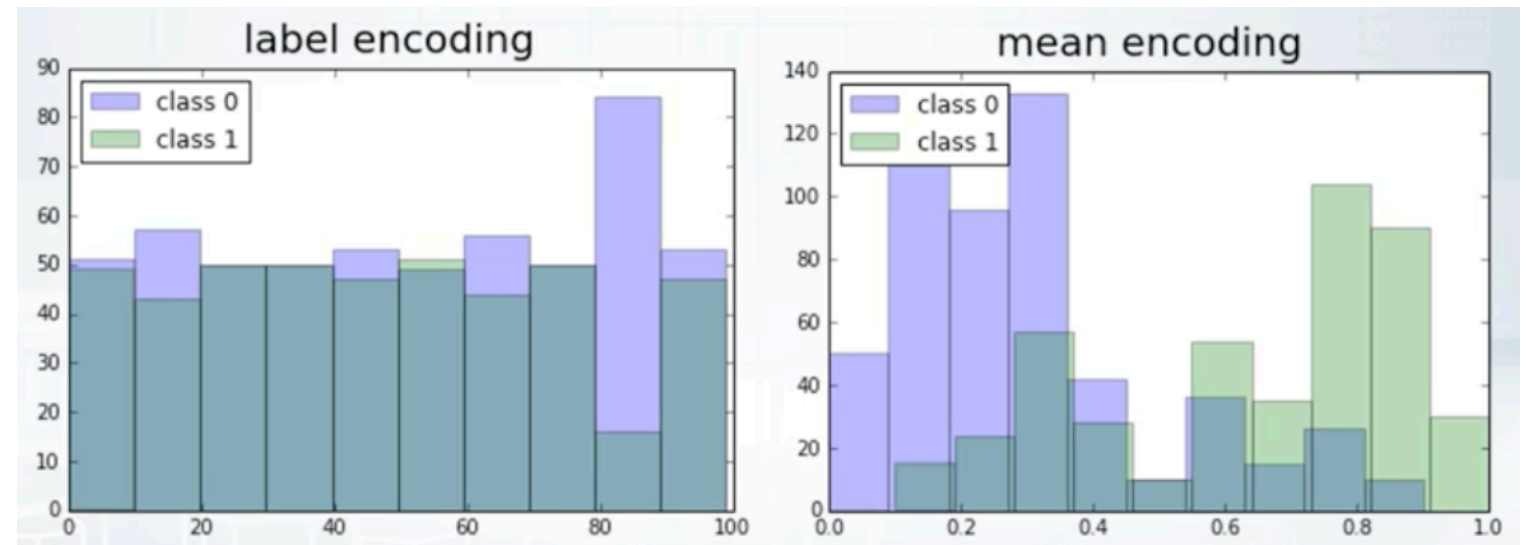
sales = downcast(sales)
sales = sales.sort_values(['date_block_num', 'shop_id', 'item_id'])
```

Feature engineering

Mean encodings

Creare features usando l'informazione contenuta nella variabile target

	feature	feature_label	feature_mean	target
0	Milan	0	0.4	0
1	Milan	0	0.4	1
2	Milan	0	0.4	1
3	Milan	0	0.4	0
4	Milan	0	0.4	0
5	Rome	1	0.8	1
6	Rome	1	0.8	1
7	Rome	1	0.8	1
8	Rome	1	0.8	0
9	Venice	2	0.0	0
10	Venice	2	0.0	0
11	Rome	1	0.8	1



Feature engineering

Mean encodings

⚠ Pericolo Overfitting ⚠

- Calcolare mean encodings **prima** di effettuare il train/validation split (la feature potrebbe comparire in entrambi i dataset con lo stesso valore ma con diversi valori target, risultando in “target means” completamente diverse tra train e validation set).
- Applicare qualche forma di regolarizzazione:
 - Cross Validation
 - Leave-One-Out
 - Expanding Mean

Per approfondire: <https://www.kaggle.com/dustinthewind/making-sense-of-mean-encoding>

Feature engineering

Mean encodings - Cross Validation

1. Divido il dataset in `n_splits` folds
2. Per ciascun split (`X_tr`, `X_val`), calcolo i mean encodings usando solo i dati che si trovano in `X_tr`.
3. Sui dati che si trovano in `X_val` vado a mappare i valori calcolati

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=123)

for tr_idx, val_idx in skf.split(X=np.zeros(n_samples), y=df['target'].values):
    X_tr, X_val = df.iloc[tr_idx], df.iloc[val_idx]
    for col in cols: # cols contains the columns we want to encode
        means = X_tr.groupby(col)['target'].mean()
        X_val[col+'_mean_target'] = X_val[col].map(means)
    train_new.iloc[val_idx] = X_val

global_mean = df['target'].mean()
train_new.fillna(global_mean, inplace=True) # fill NaNs with global mean
df = pd.concat([df, train_new], axis=1)
```


Feature engineering

Ritardi

Schema generale

```
feature_to_lag = ['item_cnt_month']
df_shifted = df[['date_block_num', 'shop_id', 'item_id'] + feature_to_lag].copy()

lag = 1
df_shifted.loc[:, 'date_block_num'] = df_shifted['date_block_num'] + lag
df_shifted.rename(columns = {'item_cnt_month': 'item_cnt_month_lag_1'}, inplace=True)

cols_to_merge = ['date_block_num', 'shop_id', 'item_id', 'item_cnt_month_lag_1']
df = df.merge(df_shifted[cols_to_merge], on=['date_block_num', 'shop_id', 'item_id'], how='left')
df.drop(columns = feature_to_lag, inplace=True)
```

	shop_id	item_id	date_block_num	item_cnt_month	item_cnt_month_lag_1
0	1	22	9	11.0	NaN
1	1	22	10	31.0	11.0
2	1	22	11	23.0	31.0
3	1	22	12	15.0	23.0

Dataset finale

Int64Index: 8033580 entries, 827691 to 8861270

Data columns (total 46 columns):

#	Column	Dtype
0	shop_id	int16
1	item_id	int16
2	date_block_num	int8
3	item_cnt_month	int16
4	item_name	int16
5	item_category_id	int16
6	item_name_2	int16
7	item_name_3	int16
8	months_since_release	int8
9	section	int16
10	cat_name	int16
11	cat_sub_name	int16
12	cat_sec_sub_name	int16
13	shop_name	int16
14	shop_city	int16
15	shop_type	int16
16	shop_first_month	int16
17	shop_last_month	int16
18	year	int16
19	month	int16
20	month_length	int8
21	weekend_days	int8

Int64Index: 8033580 entries, 827691 to 8861270

Data columns (total 46 columns):

#	Column	Dtype
22	new	bool
23	months_since_opening	int8
24	item_name_age_cat_mean	float32
25	item_cnt_month_lag_1	float32
26	item_cnt_month_lag_2	float32
27	item_cnt_month_lag_3	float32
28	item_shop_avg_price_lag_1	float32
29	item_shop_avg_price_lag_2	float32
30	item_shop_avg_price_lag_3	float32
31	item_mean_lag_1	float32
32	item_mean_lag_2	float32
33	item_mean_lag_3	float32
34	item_name_mean_lag_1	float32
35	item_name_mean_lag_2	float32
36	item_name_mean_lag_3	float32
37	item_city_mean_lag_1	float32
38	item_city_mean_lag_2	float32
39	item_city_mean_lag_3	float32
40	shop_category_mean_lag_1	float32
41	shop_category_mean_lag_2	float32
42	shop_category_mean_lag_3	float32
43	qmean	float32
44	ratio_1	float32
45	ratio_2	float32

dtypes: bool(1), float32(22), int16(18), int8(5)

memory usage: 1.0 GB

Modello

Apprendimento ensemble

- Bagging
- **Boosting**
- Stacking



dmlc
XGBoost

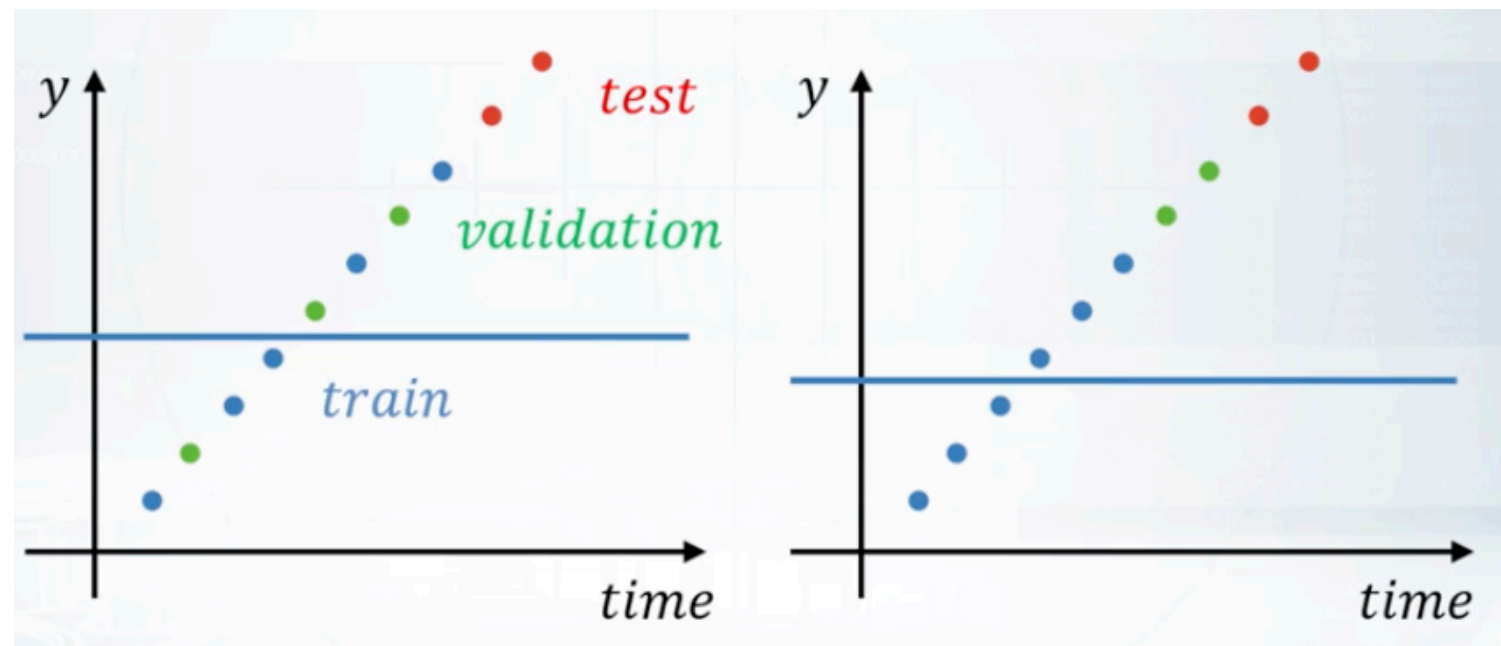
 **LightGBM**

 **CatBoost**

Modello

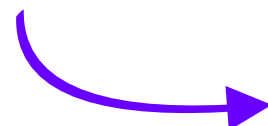
Train/validation split

Lo split in train/validation set deve rispecchiare lo split train/test set.



Un modello allenato su un train/validation split del primo tipo, cercherebbe di stimare le vendite relative ai mesi mancanti interpolando dati passati e futuri. Non sarebbe in grado di fare previsioni sul test set poiché non dispone di dati futuri.

Un modello allenato su un train/validation split del secondo tipo, cercherebbe di tener conto del trend dei dati passati



Validation set: Ottobre 2015

Analisi

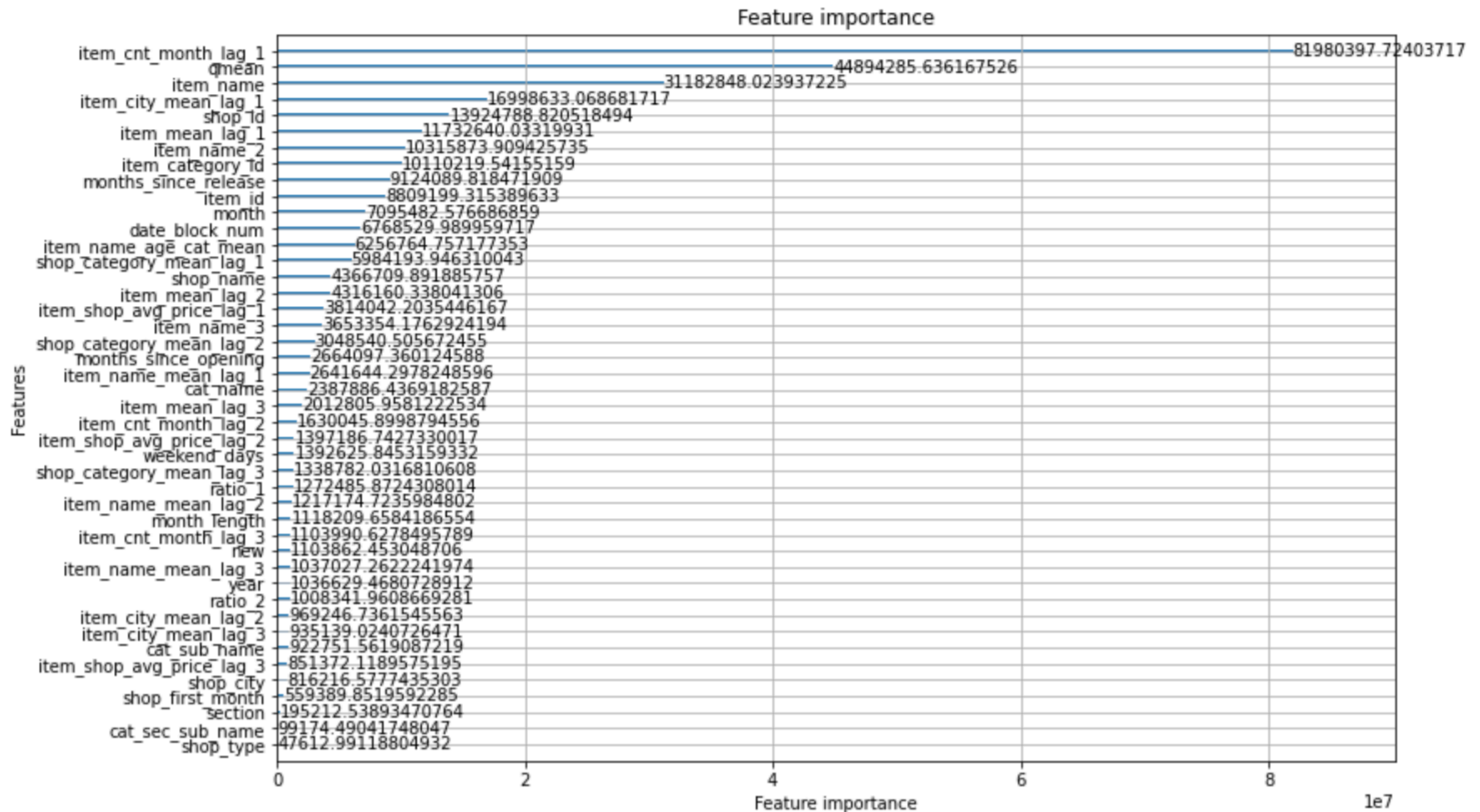
Risultati

```
Training until validation scores don't improve for 150 rounds.  
[100]   training's rmse: 0.838483      valid_1's rmse: 0.832948  
[200]   training's rmse: 0.721924      valid_1's rmse: 0.78841  
[300]   training's rmse: 0.674041      valid_1's rmse: 0.782199  
[400]   training's rmse: 0.647253      valid_1's rmse: 0.779913  
[500]   training's rmse: 0.627835      valid_1's rmse: 0.779727  
[600]   training's rmse: 0.613665      valid_1's rmse: 0.779838  
[700]   training's rmse: 0.602582      valid_1's rmse: 0.77997  
Early stopping, best iteration is:  
[554]   training's rmse: 0.61981      valid_1's rmse: 0.779485
```

RMSE Test set: 0.87590  TOP 4%

Analisi

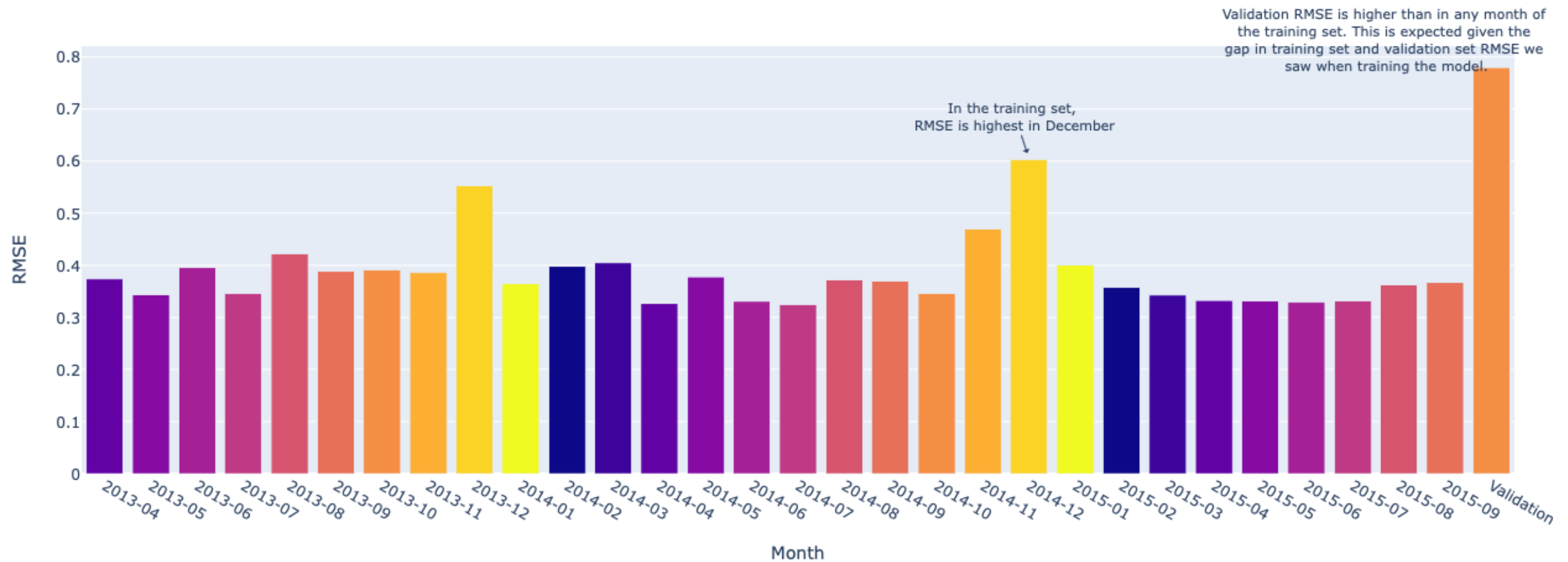
Importanza delle features



Analisi

Root Mean Squared Error

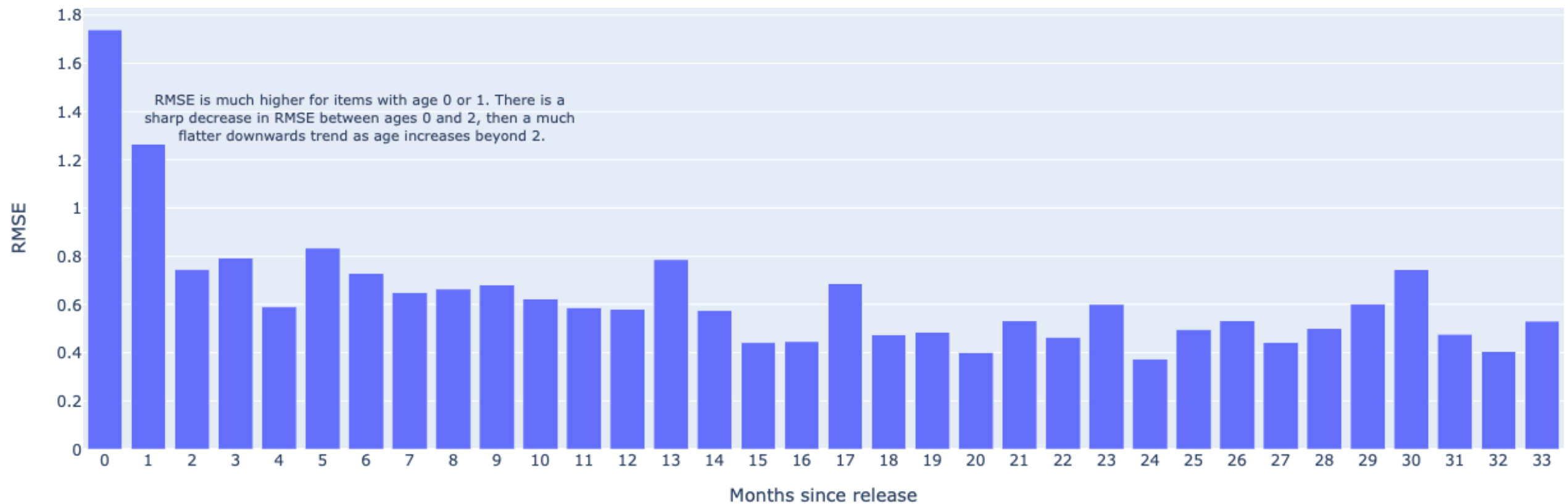
RMSE by Month



Analisi

RMSE in base all'età del prodotto

Validation set RMSE by Item Age

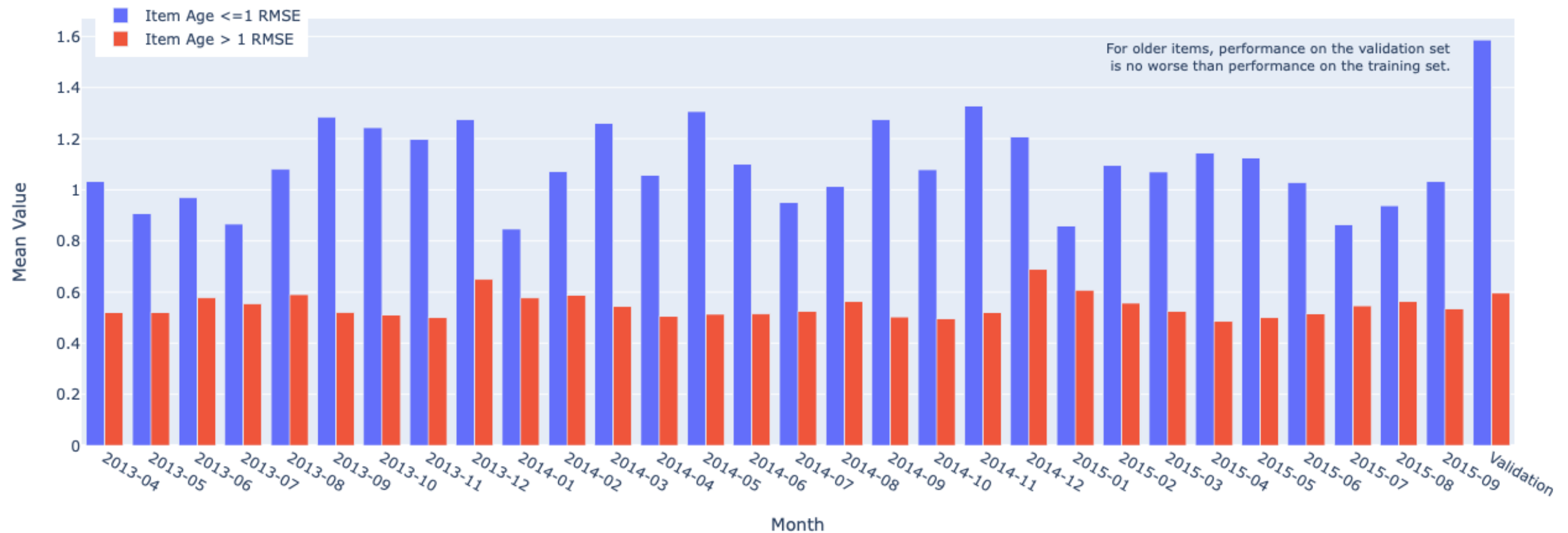


Commettiamo un errore maggiore con i prodotti appena rilasciati (primi due mesi).

Analisi

Confronto nuovi prodotti vs. vecchi

RMSE by Month for New and Old Items



Rispetto ai prodotti presenti da più di due mesi, i prodotti nuovi causano un errore decisamente maggiore.

Potremmo addestrare due modelli: uno per i nuovi prodotti, uno per gli altri.

Analisi

Categorie / Shop

Per ogni categoria visualizzo gli shop che vendono prodotti appartenenti ad essa

item_category_id	(shop_id, nunique)	(shop_id, unique)	(item_cnt_month, sum)	item_category_name	test_set_items
0	1	[38]	1.0	PC - Гарнитур/Наушники	1
44	1	[55]	252.0	Книги - Аудиокниги (Цифра)	7
36	1	[55]	22.0	Карты оплаты - Windows (Цифра)	2
34	1	[55]	1554.0	Карты оплаты - Live! (Цифра)	5
31	1	[55]	24332.0	Игры PC - Цифра	277
26	1	[55]	107.0	Игры Android - Цифра	2
74	1	[55]	59.0	Программы - MAC (Цифра)	1
76	1	[55]	3906.0	Программы - Для дома и офиса (Цифра)	61
27	1	[55]	8.0	Игры MAC - Цифра	1
54	1	[55]	6063.0	Книги - Цифра	38
9	1	[12]	3179.0	Доставка товара	8
78	1	[55]	2510.0	Программы - Обучающие (Цифра)	49
45	34	[6, 10, 16, 18, 21, 22, 24, 25, 26, 28, 31, 37...	3355.0	Книги - Аудиокниги 1С	35
--	--	-----	-----	..	---

Molte categorie sono vendute solo dallo shop 55

Analisi

Confronto negozi

	sq_err	target	lgb_pred	test_pred	shop_name
shop_id					
25	2.722791	0.999621	0.826498	0.978392	Москва ТРК "Атриум"
31	2.341101	1.013642	0.977070	1.106327	Москва ТЦ "Семеновский"
42	2.169030	0.675445	0.553410	0.641385	СПб ТК "Невский Центр"
28	1.225780	0.521410	0.538922	0.613245	Москва ТЦ "МЕГА Теплый Стан" II
55	0.914638	0.306934	0.271807	0.301643	Цифровой склад 1С-Онлайн
21	0.720978	0.330807	0.272037	0.321805	Москва МТРЦ "Афи Молл"
47	0.715364	0.306366	0.292491	0.329278	Сургут ТРЦ "Сити Молл"

- Lo shop 55 è il 5° shop per errore quadratico medio sul validation set.
- Era l'unico a vendere certe categorie.
- Analizzare il tipo di prodotti venduti per valutare se abbia adottato un particolare business model (e.g. online shop)
- Addestrare un modello specifico per questo shop?

Grazie per l'attenzione

EDA + Feature Engineering (Predict Future Sales)

<https://colab.research.google.com/drive/1LAImI5mQZEdP0JpQU8AUndb7u-jsi-uB?usp=sharing>

Model Training + Evaluation (Predict Future Sales)

https://colab.research.google.com/drive/1sdxvVosUzO6lUGhuoU-T-2W3_HJQ6qcH?usp=sharing

Appunti su EDA e Feature Engineering

<https://github.com/pietroventurini/machine-learning-notes/blob/main/2.1%20-%20Data%20Analysis.ipynb>

Pietro Venturini · Università degli Studi di Brescia · 2021